



INFORME TÉCNICO

Pipelines Compartidas con GitHub Actions

Validación Desacoplada y Control de Release

Asignatura:	AUY1104 — Ciclo de Vida del Software II
Profesor:	ANDRES PATRICIO SANCHEZ OSSANDON
Estudiante:	Alonso Esteban Mieres Fuentes
Repositorio Cliente:	github.com/alonsoM10/AUY1104-SharedClient
Repositorio Central:	github.com/alonsoM10/AUY1104-SharedCentral
Tag evaluado:	v1.0.5
Fecha:	Abril 2026

1. Resumen Ejecutivo

El presente informe documenta la implementación de una arquitectura de pipelines CI/CD compartidos para TechMaster LTDA, en el contexto de la asignatura AUY1104. La solución aborda la brecha crítica identificada: la ausencia de una validación desacoplada del proceso de publicación de software.

Se implementaron dos pipelines diferenciados en el repositorio cliente: uno de validación continua mediante pruebas automatizadas con Jest, y otro de release que publica imágenes Docker versionadas en Docker Hub. Ambos consumen la lógica centralizada en AUY1104-SharedCentral, aplicando el principio DRY a la infraestructura CI/CD. La validación ocurre en cada push de desarrollo, mientras que la publicación solo se activa ante un tag de versión semántica, garantizando que ninguna imagen llegue a producción sin haber pasado por los tests.

2. Arquitectura de la Solución

La arquitectura sigue el patrón proveedor-consumidor de GitHub Actions Reusable Workflows, con dos repositorios de roles definidos:

- AUY1104-SharedCentral (Proveedor): Contiene `deploy-api.yaml` con dos jobs encadenados: `deps-and-test` (`npm install + npm test`) y `build-and-push` (Docker build + push). Es la fuente de verdad para la lógica CI/CD.
- AUY1104-SharedClient (Consumidor): Contiene la aplicación Node.js/Express, los tests con Jest, el Dockerfile y los dos workflows que determinan cuándo y cómo se invoca la receta central.

Los secretos `DOCKER_USERNAME` y `DOCKER_PASSWORD` se almacenan en GitHub Secrets del repositorio cliente y se pasan explícitamente al workflow central mediante la interfaz `workflow_call.secrets`, garantizando el principio de mínimo privilegio.

3. Flujo de Pipelines (push vs tag)

Pipeline	Trigger	Archivo	Acción
Release	push tag v*.*.*	client.yaml	Tests + Build + Push Docker Hub
Tests	push en ramas (no main, no tags)	tests.yaml	npm install + npm test

Pipeline de Tests (tests.yaml)

Se activa con cualquier push a ramas que no sean main ni correspondan a un tag, usando `branches-ignore: [main]` y `tags-ignore: [***]`. Su único propósito es ejecutar `npm install` y `npm test`.

sin construir imágenes Docker. Corre directamente en ubuntu-latest sin invocar el repositorio central, ya que no requiere credenciales de Docker Hub.

Pipeline de Release (client.yaml)

Se activa exclusivamente con tags `v*.*.*`. Invoca `deploy-api.yaml` del repositorio central vía `workflow_call`, pasando el nombre de imagen, el tag (`github.ref_name`) y los secretos. El encadenamiento `needs: deps-and-test` garantiza que `build-and-push` solo corre si los tests pasan, por lo que ninguna imagen llega a Docker Hub si hay fallos.

4. Decisiones de Diseño

Separación de responsabilidades

Se mantienen dos workflows independientes en lugar de uno que haga todo, aplicando el principio de responsabilidad única a los pipelines. Esta decisión reduce el riesgo operacional: un fallo en los tests no bloquea ramas activas, y un error en Docker Hub no afecta la validación continua. Cada pipeline puede evolucionar de forma independiente.

Centralización en repositorio proveedor

La lógica Docker (login, build, push) se centraliza en SharedCentral mediante `workflow_call`. Si se necesita actualizar la versión de Node, agregar un escáner de vulnerabilidades o cambiar el registry, el cambio se realiza en un único lugar y todos los proyectos consumidores lo heredan automáticamente, alineado con el principio DRY.

Control de ejecución por eventos

El uso de tags SemVer como trigger del release conecta el ciclo de vida de Git con Docker Hub: solo las versiones intencionalmente etiquetadas generan imágenes. La exclusión explícita de main y tags en `tests.yaml` evita superposición de ejecuciones y consumo innecesario de minutos de GitHub Actions.

Gestión de secretos

Se optó por el paso explícito de secretos (nombre por nombre) en lugar de `secrets: inherit`, garantizando que el repositorio central solo recibe exactamente las credenciales que necesita, reduciendo la superficie de exposición.

5. Relación entre Validación y Release

El diseño establece una dependencia unidireccional: el release siempre valida, pero la validación no siempre hace release. En el pipeline de release, el job `build-and-push` declara `needs: deps-and-test`, lo que garantiza a nivel de infraestructura que ninguna imagen Docker se construye si los tests fallan.

El pipeline de tests corre de forma independiente en desarrollo, entregando retroalimentación rápida sin el overhead del proceso Docker. La trazabilidad se mantiene mediante el tag Git: cuando se crea v1.0.5, el mismo commit que pasó los tests es el que se empaqueta en la imagen Docker etiquetada v1.0.5 en Docker Hub. Lo que se prueba es exactamente lo que se despliega.

6. Alcance y Limitaciones

Alcance

- Dos pipelines diferenciados con control preciso de triggers por rama y tag.
- Lógica CI/CD centralizada en repositorio proveedor reutilizable con Reusable Workflows.

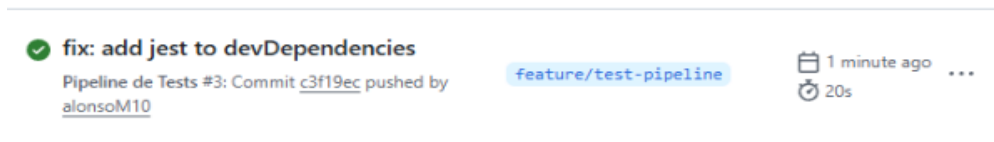
- Tests automatizados con Jest como requisito obligatorio del release.
- Publicación de imágenes Docker versionadas con etiquetas SemVer y latest.
- Gestión segura de credenciales con paso explícito entre repositorios.

Limitaciones

- No se implementaron ambientes diferenciados (dev/qa/prod) con aprobaciones manuales.
- El Dockerfile no usa multi-stage build para optimizar el tamaño de la imagen final.
- En redes con restricciones (laboratorios), el push a Docker Hub puede sufrir timeouts que requieren reintento manual.

7. Evidencia de Ejecución

Pipeline de Tests — rama feature/test-pipeline (verde):



Pipeline de Release — tag v1.0.5 (verde, ambos jobs):

